Programa en ensamblador

Snake Game

Nota

Estudiante	Escuela	Asignatura
Camilo B.A. Capatinta Almanza (Teoría grupo B) ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Lab. Arquitectura de Computadoras Semestre: IV Código: 1702227

Profesor	Proyecto	Duración
Pedro A. Rodriguez Gonzales	Snake Game	02 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	20/10/2026	–/10/2026

Índice

1. Actividades

1.1. Descripción de la actividad

El curso de Arquitectura de Computadoras de la carrera de Ingeniería de Sistemas de la Universidad Nacional de San Agustín de Arequipa (UNSA) encarga a cada estudiante un trabajo integrador que consiste en programar en ensamblador un juego. El enunciado del profesor exige que el programa aplique, en un contexto reconocible y motivador, los contenidos revisados en las prácticas del curso: manipulación de datos, modos de direccionamiento, control de flujo, aritmética y uso de la pila.

Este documento describe el proyecto desarrollado para cumplir ese encargo: una implementación del clásico juego **Snake** para procesador 8086, escrita para el emulador EMU8086 y ensamblada como programa de formato COM. El documento se limita a describir lo que el código fuente `snake.asm` contiene; no propone cambios al programa ni atribuye funcionalidades que este no posea. Cuando un dato no está definido con certeza, el texto lo indica y la sección *Notas para el autor*, al final del documento, lo enumera como pendiente de confirmar.

1.2. Objetivos

El objetivo general es construir un juego funcional en ensamblador 8086 y documentarlo con rigor técnico. Los objetivos específicos son los siguientes:

- Aplicar la **Práctica 2** (sumador total): operaciones aritméticas y lógicas básicas con acarreo, `ADD`, `ADC`, `SUB`, `XOR`, `AND` y desplazamientos.
- Aplicar la **Práctica 3** (modos de direccionamiento): direccionamiento inmediato, de registro, directo e indirecto por registro.
- Aplicar la **Práctica 4** (control de flujo): comparaciones, saltos condicionales e incondicionales, bucles con `LOOP` y llamadas a procedimientos con `CALL`/`RET`.
- Aplicar la **Práctica 5** (operaciones aritméticas y pila): `MUL`, `DIV`, `INC`, `DEC`, `PUSH`, `POP`, constantes con `EQU` y datos con `DB`/`DW`.
- Utilizar los servicios del BIOS (`INT 10h`, `INT 16h`, `INT 1Ah`) y de DOS (`INT 21h`) para pantalla, teclado y temporización.
- Documentar el diseño, el funcionamiento, el rendimiento y las limitaciones del programa para su exposición en clase.

1.3. Entorno de desarrollo y herramientas

El desarrollo y la verificación se realizan con las herramientas de la Tabla **??**.

Cuadro 1: Herramientas empleadas en el proyecto.

Herramienta	Uso
EMU8086 (versión pendiente de confirmar)	Edición, ensamblado y emulación del programa en formato COM.
Editor de texto	Edición alternativa del código fuente `snake.asm`.
DOSBox (opcional)	Ejecución alternativa del archivo COM generado.

1.4. Alcance y entregables

El alcance comprende un único juego de un jugador, con tablero fijo de 40 columnas por 10 filas, sin pausa, sin sonido y sin reinicio. Los entregables son tres:

1. `snake.asm`: código fuente completo del juego.
2. `descripcion.md`: descripción breve del proyecto.
3. Este documento (`Snake_Game.tex`), que documenta el diseño y la implementación.

2. Marco teórico

2.1. Arquitectura 8086 y segmentación

El procesador 8086 es un microprocesador de 16 bits con bus de direcciones de 20 bits. Como sus registros solo almacenan 16 bits, la dirección física se forma combinando un registro de segmento con un desplazamiento (*offset*): la dirección física equivale al segmento multiplicado por 16 más el desplazamiento. Los registros de segmento son `CS` (código), `DS` (datos), `SS` (pila) y `ES` (extra).

Las banderas del registro de estado resultan esenciales para este proyecto. La bandera `ZF` (**Zero Flag**) indica que el resultado de la última operación fue cero o que una comparación resultó en igualdad. La bandera `CF` (**Carry Flag**) indica un acarreo o un préstamo en operaciones sin signo. Estas dos banderas deciden todos los saltos condicionales del programa (véase la Tabla **??**).

Cuadro 2: Banderas que deciden los saltos condicionales del código.

Salto	Bandera	Interpretación
`JE` / `JNE`	`ZF`	Igualdad o desigualdad tras `CMP`.
`JB` / `JAE`	`CF`	Comparación *sin signo*: menor que / mayor o igual que.

Las instrucciones `JL`/`JGE`, que interpretan los operandos con signo, **no se utilizan** en este código, por la razón que explica la sección **??**.

2.2. Modelo de memoria del formato COM

Un programa COM se carga en un único segmento y ocupa como máximo 64 KB. El cargador de DOS inicializa `CS`, `DS`, `ES` y `SS` con el mismo valor, de modo que el programa no necesita inicializar `DS` manualmente: código, datos y pila comparten segmento desde el arranque. Por esa razón el programa no declara `.model` ni `.stack`, y comienza con la directiva `org 100h`, que reserva los primeros 256 bytes del segmento para el prefijo de segmento de programa (PSP).

Comparación con signo y sin signo. Tras `CMP`, las instrucciones `JL`/`JGE` interpretan el resultado como número con signo, mientras que `JB`/`JAE` lo interpretan sin signo. En el juego, la fila y la columna de la cabeza se mantienen en registros de 8 bits. Cuando la cabeza está en la fila 0 y se mueve hacia arriba, `DEC` deja el valor `255` (`0FFh`). Con interpretación con signo, `0FFh` equivale a -1 y `JGE` lo trataría como menor que el límite del tablero, de modo que no detectaría la pared. Con interpretación sin signo, `255` es mayor que cualquier límite válido y `JAE` detecta correctamente la salida del tablero. Por eso la comparación sin signo es obligatoria en este programa.

2.3. Modo texto 80x25 y servicios BIOS (INT 10h)

En modo texto de 80 columnas por 25 filas, la memoria de video comienza en la dirección `B800:0000`; cada celda ocupa dos bytes (carácter y atributo de color). Escribir directamente en esa memoria sería la opción más rápida. Este prototipo emplea en cambio la interrupción `INT 10h` del BIOS, que es más sencilla de explicar y de verificar: es una decisión didáctica, no óptima. Los servicios utilizados se resumen en el Anexo C (Tabla **??**). Los atributos de color usados son `07h` (gris, borde), `0Ah` (verde, serpiente) y `0Ch` (rojo, comida).

2.4. Lectura de teclado (INT 16h) y reloj del BIOS (INT 1Ah)

La función `AH=01h` de `INT 16h` consulta el búfer de teclado *sin bloquear*: indica mediante `ZF` si hay una tecla pendiente. La función `AH=00h` extrae la tecla del búfer; para teclas extendidas, como las flechas, devuelve `AL=0` y el código de exploración (*scancode*) en `AH`.

La función `AH=00h` de `INT 1Ah` devuelve el contador de ticks del reloj del BIOS en `CX:DX`. Este contador avanza aproximadamente 18,2 veces por segundo; el procedimiento `DELAY` lo emplea como base de tiempo.

2.5. Servicios DOS (INT 21h)

La interrupción `INT 21h` ofrece los servicios del sistema operativo. En este programa interviene en la impresión de caracteres o cadenas, en particular en `PRINT_NUM`, que solo se ejecuta cuando la serpiente come. El detalle de las funciones empleadas se resume en el Anexo C.

2.6. Cola circular y direccionamiento indirecto

Una **cola circular** es un arreglo de tamaño fijo con dos índices: uno de escritura (`head`) y otro de lectura (`tail`). Al llegar al final del arreglo, el índice vuelve a cero. En el juego, la cola almacena los índices de las celdas ocupadas por la serpiente, de la cola (más antigua) a la cabeza (más reciente). Mover la serpiente consiste en insertar una celda en `head` y, si no comió, retirar la celda de `tail`. Con un arreglo lineal habría que desplazar todos los elementos tras cada movimiento (`memmove`); la cola circular lo evita y mantiene cada operación en tiempo constante.

El acceso al arreglo `queue`, de palabras de 16 bits, usa direccionamiento indirecto por registro (`[SI]`), con el índice multiplicado por dos mediante `SHL SI,1`. El acceso al arreglo `board`, de bytes, usa `[BX]`.

3. Diseño del juego

3.1. Reglas y controles

El jugador controla una serpiente que se mueve sin detenerse. Las reglas son las siguientes:

- La serpiente comienza con tres segmentos en la fila 5, columnas 19, 20 y 21 del tablero, con la cabeza en la columna 21 y orientada a la derecha.
- Al ocupar la celda de la comida, la serpiente crece un segmento y el puntaje aumenta en uno; aparece una nueva comida en una celda libre.
- El juego termina al chocar con una pared o con el propio cuerpo. La celda que ocupa la cola en ese instante *cuenta como colisión*.
- El giro de 180° está prohibido: se rechaza la tecla cuya dirección sea opuesta a la actual.
- La tecla `ESC` termina el programa en cualquier momento y restaura el modo de video y el cursor.

La Tabla **??** resume los controles. La dirección se codifica con `dir`: 0 arriba, 1 derecha, 2 abajo, 3 izquierda.

3.2. Representación del tablero

El tablero lógico tiene 40 columnas por 10 filas (400 celdas) y se guarda en el arreglo `board`, con un byte por celda: `0` libre, `1` serpiente y `2` comida. El índice de una celda es fila $\times$ 40 + columna. La celda de fila r y columna c se dibuja en la pantalla en la fila $r+2$ y la columna $c+20$ del modo texto. Por ejemplo, la celda de la fila 5 y la columna 19 tiene índice $5 \cdot 40 + 19 = 219$. El borde se dibuja en gris (`07h`) alrededor del tablero; el puntaje aparece en la fila 0, columna 19, con el formato “Puntaje: ” seguido del número.

3.3. Estructuras de datos

La Tabla **??** lista las constantes y variables del programa. Su contenido coincide con el código fuente y se repite como Anexo B.

Cuadro 3: Teclas aceptadas, códigos y dirección resultante.

Tecla	ASCII (`AL`)	Scancode (`AH`)	Dirección	`dir`
Flecha arriba	`0`	`48h`	Arriba	0
Flecha derecha	`0`	`4Dh`	Derecha	1
Flecha abajo	`0`	`50h`	Abajo	2
Flecha izquierda	`0`	`4Bh`	Izquierda	3
`W` / `w`	`57h` / `77h`	—	Arriba	0
`D` / `d`	`44h` / `64h`	—	Derecha	1
`S` / `s`	`53h` / `73h`	—	Abajo	2
`A` / `a`	`41h` / `61h`	—	Izquierda	3
`ESC`	`1Bh` (27)	—	Salir	—

Cuadro 4: Constantes y variables del programa.

Nombre	Tipo	Tamaño	Uso
`BOARD_W`	`EQU`	—	Ancho del tablero: 40.
`BOARD_H`	`EQU`	—	Alto del tablero: 10.
`BOARD_SIZE`	`EQU`	—	Total de celdas: 400.
`board`	`DB`	400 bytes	Estado de cada celda (0, 1, 2).
`queue`	`DW`	400 palabras	Cola circular de índices de la serpiente.
`head`	`DW`	1 palabra	Índice de escritura de la cola (cabeza).
`tail`	`DW`	1 palabra	Índice de lectura de la cola (cola).
`dir`	`DB`	1 byte	Dirección actual (0 a 3).
`headRow`	`DB`	1 byte	Fila de la cabeza.
`headCol`	`DB`	1 byte	Columna de la cabeza.
`score`	`DW`	1 palabra	Puntaje.
`seed`	`DW`	1 palabra	Semilla del generador pseudoaleatorio.
`delayTicks`	`DB`	1 byte	Ticks de espera por fotograma (valor 2).
`msgScore`	`DB`	cadena	Texto “Puntaje: ”.
`msgGameOver`	`DB`	cadena	Texto “GAME OVER!”.

3.4. Procedimientos y sus contratos

El programa define exactamente diez procedimientos: `INIT_GAME`, `DRAW_BORDER`, `READ_KEY`, `MOVE_SNAKE`, `CHECK_COLLISION`, `SPAWN_FOOD`, `DRAW_CELL`, `PRINT_NUM`, `DELAY` y `GAME_OVER`. La Tabla **??** presenta la ficha de cada uno. Cada procedimiento guarda con `PUSH` los registros que modifica y los restaura con `POP` antes de `RET`; la lista exacta de registros de cada uno está pendiente de confirmar contra el listado completo (Anexo A) y se señala en las notas finales. No se emplean `PUSHA` ni `POPA`.

Cuadro 5: Fichas de los procedimientos.

Procedimiento	Entradas	Salidas	Resumen
`INIT_GAME`	Ninguna (usa constantes).	Variables y `board` inicializados.	Limpia el tablero, coloca los tres segmentos iniciales, fija `dir`, `headRow`, `headCol`, `head`, `tail` y puntaje.
`DRAW_BORDER`	Ninguna.	Borde dibujado.	Dibuja el marco gris del tablero mediante bucles con `LOOP`.

Procedimiento	Entradas	Salidas	Resumen
`READ_KEY`	Estado del búfer de teclado.	`AL`: 0 si el juego continúa (ver Sección ??). Actualiza `dir`.	Consulta sin bloquear, consume la tecla, detecta `ESC`, flechas y WASD, y rechaza el giro de 180°.
`MOVE_SNAKE`	`dir`, `headRow`, `headCol`, `head`, `tail`.	`AL=1` si hubo colisión; actualiza cola y tablero.	Calcula la nueva cabeza, consulta la colisión, encola la nueva celda y, si no come, libera la cola.
`CHECK_COLLISION`	Fila y columna candidatas.	`AL=1` si choca con pared o cuerpo.	Compara con límites mediante `JAE`, calcula el índice con `MUL` y consulta `board`.
`SPAWN_FOOD`	`seed`, `board`.	Comida en una celda libre (o ninguna, tras 1000 intentos).	Genera posiciones pseudoaleatorias hasta hallar una celda libre.
`DRAW_CELL`	Índice de celda, carácter y atributo.	Celda dibujada.	Convierte índice a fila y columna con `DIV`, posiciona el cursor y escribe el carácter con `INT 10h`.
`PRINT_NUM`	Número (puntaje).	Número impreso.	Descompone el número en dígitos con `DIV` y los imprime con `INT 21h`.
`DELAY`	`delayTicks`.	Ninguna.	Espera el número de ticks indicado leyendo `INT 1Ah`.
`GAME_OVER`	`score`.	Mensaje y puntaje final.	Muestra “GAME OVER!” y el puntaje en la fila 14 y espera una tecla.

4. Desarrollo e implementación

4.1. Altas y bajas del diseño

El diseño evolucionó mediante decisiones de incorporación (altas) y de descarte (bajas). Esta subsección las narra y justifica.

Altas. *Cola circular.* Se adoptó una cola circular de índices para representar la serpiente. La alternativa, un arreglo lineal con desplazamiento de elementos en cada paso, habría exigido copiar hasta 400 palabras por movimiento. La cola circular reduce el costo a una inserción y, como máximo, una extracción.

Separación entre `CHECK_COLLISION` y el manejo de comida. La verificación de choque solo informa si la celda destino es mortal; la decisión de comer, crecer o liberar la cola corresponde a `MOVE_SNAKE`. Esta separación mantiene cada procedimiento con una responsabilidad única y facilita la explicación en clase.

Dibujo selectivo. En lugar de redibujar el tablero completo en cada fotograma, el programa actualiza únicamente la nueva cabeza, la celda de la cola cuando esta se libera y la comida al generarse.

Bajas. *Redibujado completo del tablero.* Se descartó porque provocaba parpadeo visible y un costo innecesario: cada fotograma habría emitido cientos de llamadas a `INT 10h`.

Tabla precalculada de fila $\times 40$. Se descartó porque añadía complejidad de datos y de acceso sin aportar una mejora relevante en un prototipo cuyo cuello de botella es el retardo deliberado. El índice se calcula con `MUL`, lo que además permite ejercitar la Práctica 5.

Sonido y pausa. Quedaron fuera del alcance: no forman parte de los objetivos del curso y habrían incrementado el tamaño y la superficie de pruebas del programa.

4.2. Decisiones de implementación destacadas

- **Comparación sin signo.** La detección de paredes usa `CMP` con `JAE`, porque `DEC` sobre una fila o columna en cero produce `255` (véase el Marco teórico).
- **Giro de** 180°**.** `READ_KEY` rechaza la nueva dirección si es igual a (`dir` + 2) `AND` 3, expresión que obtiene la dirección opuesta en aritmética módulo 4.
- **Generador pseudoaleatorio.** `SPAWN_FOOD` aplica la recurrencia `seed` ← `seed` · 25173 + 13849 con `MUL` de 16 bits y obtiene la posición como `seed` mód 400 con `DIV` de 16 bits, con `DX=0` previo.
- **Límite de intentos.** `SPAWN_FOOD` realiza como máximo 1000 intentos, de modo que no se cuelga si el tablero está lleno.
- **Salida limpia.** Al salir, el programa restaura el modo de video y el cursor mediante `INT 10h`, `AH=01h`, `CX=0607h`.

4.3. Tabla de rastreo: Práctica del curso a código

La Tabla **??** relaciona cada práctica con las instrucciones que realmente aparecen en el código. No se incluyen instrucciones ausentes del programa (por ejemplo, no se emplea `IMUL`). Las banderas que deciden los saltos están en la Tabla **??**.

Cuadro 6: Rastreo entre las prácticas del curso y el código de `snake.asm`.

Práctica	Instrucción / tema	Procedimiento real del código	Procedimiento del `.asm` donde aparece
2	`ADD/ADC`	Suma del incremento del generador con propagación de acarreo.	`SPAWN_FOOD`
2	`SUB`	Inicialización de valores y cálculo de ticks transcurridos.	`INIT_GAME`, `DELAY`
2	`XOR`	Puesta a cero de registros.	`INIT_GAME`, `READ_KEY`, `CHECK_COLLISION`, `PRINT_NUM`, `SPAWN_FOOD`
2	`AND 3`	Módulo 4 de la dirección opuesta.	`READ_KEY`
2	`SHL SI,1`	Conversión de índice a desplazamiento en palabras.	`MOVE_SNAKE`
3	`MOV` inmediato	Carga de constantes y coordenadas iniciales.	`INIT_GAME`, `DRAW_BORDER`
3	`MOV` registro–registro	Preparación de parámetros de pantalla.	`DRAW_CELL`
3	`MOV` directo a memoria	Actualización de variables.	`MOVE_SNAKE`, `INIT_GAME`
3	`MOV` indirecto	`[BX]` sobre `board`; `[SI]` sobre `queue`.	`CHECK_COLLISION` (`[BX]`); `MOVE_SNAKE`, `SPAWN_FOOD` (`[SI]`)
3	`org 100h`; `XCHG BX,DI`	Origen del formato COM; intercambio de registros.	Inicio del programa; `MOVE_SNAKE`
4	`CMP`, `JE`, `JNE`, `JB`, `JAE`	Decisiones de teclas, límites y estados de celda.	Todo el código
4	`JMP`	Saltos incondicionales y repetición del bucle.	`MOVE_SNAKE`, `main_loop`
4	`LOOP`	Bucles con contador.	`DRAW_BORDER`, `SPAWN_FOOD`, `PRINT_NUM`

Práctica	Instrucción / tema	Procedimiento real del código	Procedimiento del `.asm` donde aparece
4	`CALL/RET`	Invocación de procedimientos.	Todos los procedimientos
5	`MUL` de 8 bits	Cálculo de fila $\times 40 +$ columna.	`CHECK_COLLISION`
5	`MUL` de 16 bits	Multiplicación de la semilla.	`SPAWN_FOOD`
5	`DIV` de 8 bits	Conversión de índice a fila y columna.	`DRAW_CELL`
5	`DIV` de 16 bits	Dígitos decimales; resto módulo 400.	`PRINT_NUM`, `SPAWN_FOOD`
5	`INC/DEC`	Desplazamiento de la cabeza.	`MOVE_SNAKE`
5	`PUSH/POP`	Preservación de registros.	Todos los procedimientos
5	`EQU`, `DB`, `DW`	Constantes y datos.	Segmento de datos

5. Funcionamiento

5.1. Bucle principal

El programa inicializa el juego, dibuja el borde, genera la primera comida y entra en el bucle principal. En cada iteración, el programa ejecuta, en este orden: `READ_KEY`, `MOVE_SNAKE`, verificación de `AL=1` (choque) y `DELAY`; luego repite. El orden importa: leer la tecla antes de mover garantiza que el movimiento use la dirección más reciente, y el retardo al final fija el ritmo del juego. El Listado **??** muestra el bucle en pseudocódigo.

```
INIT_GAME
DRAW_BORDER
SPAWN_FOOD
repetir:
    READ_KEY
    si READ_KEY indica salir: terminar
    MOVE_SNAKE
    si MOVE_SNAKE devolvio 1 (choque):
        GAME_OVER
        terminar
    DELAY
    volver a repetir
```

Listing 1: Bucle principal (pseudocódigo).

5.2. Flujo detallado de una tecla

La figura de texto del Listado **??** describe el recorrido desde que el usuario presiona `W` hasta que `READ_KEY` retorna 0 con la dirección actualizada.

```
Usuario presiona W
        |
        v
INT 16h AH=01h  (consulta no bloqueante)
        |
   hay tecla? --no--> retorna 0 (sin cambio de dir)
        |si
        v
INT 16h AH=00h  (consume la tecla)
        |
   AL = 27 (ESC)? --si--> senal de salida
```

```
      |no
      v
AL = 0 (tecla extendida)?
  |si                          |no
  v                             v
AH = 48h/50h/4Bh/4Dh?       AL = W/S/A/D (may. o min.)?
  |si          |no             |si           |no
  v            v               v             v
nueva dir   ignorar        nueva dir     ignorar
     \---------------  -----------------/
                    \/
     nueva == (dir+2) AND 3 ?   (giro de 180)
         |si                      |no
         v                        v
     rechazar              dir = nueva
         \--------  --------/
                  \/
           retorna 0
```

Listing 2: Flujo de una tecla en READ_KEY.

La combinación de la Tabla **??** y del Listado **??** define el comportamiento completo del teclado.

5.3. Flujo detallado de un movimiento

`MOVE_SNAKE` calcula la nueva celda de la cabeza según `dir`: incrementa o decrementa `headRow` o `headCol`. Llama a `CHECK_COLLISION`; si hay choque, retorna `AL=1`. En caso contrario consulta `board` en la nueva celda y se bifurca:

- **No come** (celda libre): encola el índice nuevo, marca `board=1`, dibuja la nueva cabeza, libera la celda de `tail` (`board=0`), la borra de pantalla y avanza `tail`.
- **Come** (`board=2`): se desvía a `ms_eat`; se detalla más adelante.

Micro-traza: presiono `W` yendo a la derecha. La traza usa marcas de tiempo ilustrativas dentro de un fotograma de unos 110 ms. Las celdas iniciales son $219 = 5 \cdot 40 + 19$, $220 = 5 \cdot 40 + 20$ y $221 = 5 \cdot 40 + 21$; la nueva cabeza será $181 = 4 \cdot 40 + 21$.

- **T=0 ms.** El usuario presiona `W`. Estado: `dir`=1, `headRow`=5, `headCol`=21, `queue`=[219, 220, 221], `head`=3, `tail`=0, `board[219]`=1, `board[181]`=0, `score`=0. La tecla permanece en el búfer del BIOS.
- **T=50 ms.** `DELAY` sigue esperando; el estado no cambia.
- **T=110 ms.** Comienza el fotograma siguiente. `READ_KEY` consume `W`; la nueva dirección es 0 y (1+2) `AND` 3 = $3 \neq 0$, de modo que no es giro de 180° y `dir` pasa de 1 a 0.
- **T≈110 ms, después de `MOVE_SNAKE`.** `headRow` pasa de 5 a 4; `headCol` permanece en 21; el índice nuevo es $181 = 4 \cdot 40 + 21$. La cola lógica pasa de [219, 220, 221] a [220, 221, 181]; `head` pasa de 3 a 4 y `tail` de 0 a 1. Además, `board[219]`=0, `board[181]`=1 y `score`=0.

5.4. Flujo detallado de "comer"

Cuando `board[idx]` vale 2, `MOVE_SNAKE` se desvía a la etiqueta `ms_eat`. Allí incrementa `score`, invoca `PRINT_NUM` para actualizar el puntaje en pantalla y `SPAWN_FOOD` para generar la siguiente comida. Se encola el índice `idx` como nueva cabeza, pero **no se avanza `tail`**: la cola no se libera y, por tanto, la serpiente crece un segmento.

5.5. Flujo detallado de un choque

Considérese la cabeza en `headRow`=0 con `dir`=0 (arriba). `DEC DH` convierte la fila candidata en `255`. `CHECK_COLLISION` compara con `BOARD_H`=10 y, al ser `JAE` una comparación sin signo, $255 \geq 10$ se cumple; la rutina devuelve `AL=1`. `MOVE_SNAKE` propaga ese valor y el bucle principal invoca `GAME_OVER`. Con `JGE`, `255` se interpretaría como -1 y no se detectaría la pared; por eso se usa `JAE`.

6. Pseudocódigo

Los siguientes listados describen cada procedimiento en lenguaje de alto nivel. Los nombres de procedimientos van en mayúsculas y las variables en minúsculas. Los listados no incluyen direcciones ni registros.

6.1. Bucle principal

Véase el Listado **??**.

6.2. INIT_GAME

```
INIT_GAME:
    poner en libre todas las celdas de board
    head = 0
    tail = 0
    dir = derecha
    score = 0
    headRow = 5
    para col desde 19 hasta 21:
        marcar board[5*40 + col] como serpiente
        encolar el indice en queue
        dibujar la celda de serpiente
    headCol = 21
```

Listing 3: Pseudocódigo de INIT_GAME.

6.3. DRAW_BORDER

```
DRAW_BORDER:
    para cada columna del marco:
        dibujar borde superior e inferior
    para cada fila del marco:
        dibujar borde izquierdo y derecho
```

Listing 4: Pseudocódigo de DRAW_BORDER.

6.4. READ_KEY

```
READ_KEY:
    si no hay tecla en el buffer:
        retornar 0
    leer y consumir la tecla
    si es ESC:
        indicar salida
    si es flecha o WASD (mayuscula o minuscula):
        nueva = direccion correspondiente
        si nueva es la opuesta de dir:
            ignorar
```

```
        si no:
            dir = nueva
    retornar 0
```

Listing 5: Pseudocódigo de READ_KEY.

6.5. CHECK_COLLISION

```
CHECK_COLLISION:
    si fila no es menor que alto del tablero:
        retornar 1
    si columna no es menor que ancho del tablero:
        retornar 1
    idx = fila * 40 + columna
    si board[idx] es serpiente:
        retornar 1
    retornar 0
```

Listing 6: Pseudocódigo de CHECK_COLLISION.

6.6. MOVE_SNAKE

```
MOVE_SNAKE:
    calcular nueva fila y nueva columna segun dir
    si CHECK_COLLISION indica choque:
        retornar 1
    idx = nueva fila * 40 + nueva columna
    si board[idx] es comida:
        score = score + 1
        PRINT_NUM
        encolar idx
        board[idx] = serpiente
        dibujar nueva cabeza
        SPAWN_FOOD
    si no:
        encolar idx
        board[idx] = serpiente
        dibujar nueva cabeza
        antigua = elemento en tail
        board[antigua] = libre
        borrar la celda antigua en pantalla
        avanzar tail circularmente
    actualizar headRow y headCol
    retornar 0
```

Listing 7: Pseudocódigo de MOVE_SNAKE.

6.7. SPAWN_FOOD

```
SPAWN_FOOD:
    repetir hasta 1000 intentos:
        seed = seed * 25173 + 13849
        pos = seed modulo 400
        si board[pos] es libre:
            board[pos] = comida
            dibujar comida
```

```
            retornar
    retornar sin generar comida
```

Listing 8: Pseudocódigo de SPAWN_FOOD.

6.8. DRAW_CELL

```
DRAW_CELL:
    fila = idx dividido 40
    columna = resto de idx dividido 40
    posicionar cursor en (fila + 2, columna + 20)
    escribir caracter con su color
```

Listing 9: Pseudocódigo de DRAW_CELL.

6.9. PRINT_NUM

```
PRINT_NUM:
    posicionar cursor tras la etiqueta de puntaje
    obtener los digitos de score dividiendo entre 10
    imprimir los digitos de izquierda a derecha
    imprimir espacios de relleno
```

Listing 10: Pseudocódigo de PRINT_NUM.

6.10. DELAY

```
DELAY:
    inicio = ticks del reloj
    mientras (ticks actuales - inicio) < delayTicks:
        consultar ticks del reloj
```

Listing 11: Pseudocódigo de DELAY.

6.11. GAME_OVER

```
GAME_OVER:
    mostrar GAME OVER! en la fila 14
    mostrar el puntaje final
    esperar una tecla
```

Listing 12: Pseudocódigo de GAME_OVER.

6.12. Pseudocódigo del programa completo

```
programa Snake:
    INIT_GAME
    DRAW_BORDER
    SPAWN_FOOD
    mientras verdadero:
        READ_KEY
        si hubo ESC:
            restaurar video y cursor
```

```
        terminar
    MOVE_SNAKE
    si hubo choque:
        GAME_OVER
        restaurar video y cursor
        terminar
    DELAY
```

Listing 13: Pseudocódigo del programa completo.

7. Análisis de rendimiento

7.1. ¿Por qué el juego es "lento"?

El tiempo de un fotograma lo domina el procedimiento `DELAY`. Con `delayTicks`=2 y un reloj del BIOS de unos 18,2 ticks por segundo, el retardo es

$$2 \times \frac{1}{18{,}2}\ \text{s} \approx 0{,}1099\ \text{s},$$

es decir, unos 110 ms, lo que equivale a unos 9 fotogramas por segundo. Esa espera representa aproximadamente el 95 % del tiempo de cada fotograma.

7.2. Presupuesto de ciclos por fotograma

La Tabla **??** resume los factores. Las instrucciones aritméticas más costosas en el 8086 son `MUL` de 8 bits (unos 70–77 ciclos), `MUL` de 16 bits (118–133), `DIV` de 8 bits (80–90) y `DIV` de 16 bits (144–162). En un fotograma normal se ejecutan aproximadamente un `MUL` y dos `DIV`, lo que suma unos 260 ciclos; como referencia, a 4,77 MHz esa cifra equivale a menos de 0,1 ms, despreciable frente a los 110 ms del retardo.

Cuadro 7: Factores de costo por fotograma.

Factor	Costo por frame	¿Cuello de botella?
`DELAY`	≈ 110 ms (2 ticks).	Sí (deliberado).
`MUL`/`DIV`	≈ 260 ciclos (1 `MUL` + 2 `DIV`).	No.
`INT 10h`	4 llamadas por fotograma en `DRAW_CELL`.	No.
`INT 16h`	0 a 2 llamadas por fotograma.	No.
`INT 1Ah`	Repetida dentro de `DELAY` mientras espera.	Parte del retardo.
`INT 21h`	Solo en `PRINT_NUM`, al comer.	No.

7.3. Cuellos de botella identificados

El único cuello de botella es el retardo programado. Las cuatro llamadas a `INT 10h` (dos por cada invocación de `DRAW_CELL`, para la nueva cabeza y para la cola borrada) y las lecturas de `INT 16h` cuestan poco frente a `DELAY`. El costo de `INT 21h` en `PRINT_NUM` solo aparece al comer y no afecta la fluidez. En conclusión, la lentitud es una decisión de diseño (`delayTicks` calibrado a ≈ 9 fps), no una limitación del 8086 ni del ensamblador.

8. Limitaciones y consideraciones

8.1. Limitaciones funcionales del juego

Cuadro 8: Limitaciones funcionales.

Limitación	Descripción
Tablero lleno sin victoria	No existe condición de victoria cuando la serpiente ocupa todo el tablero.
Celda de la cola	La celda de la cola cuenta como colisión, aunque en ese instante quedaría libre.
Sin reinicio	Tras `GAME_OVER` no se ofrece jugar de nuevo.
Sin récord	No se conserva la mejor puntuación.
Velocidad fija	`delayTicks` no cambia durante la partida.
Sin pausa	No hay tecla de pausa.
Sin sonido	No se emite ningún sonido.
Semilla repetible	La semilla puede repetirse entre ejecuciones y producir la misma secuencia de comida (condición por confirmar).

8.2. Limitaciones técnicas del código

Cuadro 9: Limitaciones técnicas.

Limitación	Descripción
`SPAWN_FOOD` falla en silencio	Tras 1000 intentos sin celda libre, retorna sin comida y sin avisar.
`DRAW_CELL`	Asume un índice de celda ≤ 399; no valida el rango.
`CHECK_COLLISION`	Es frágil ante un cambio de `JAE` a `JGE`, que dejaría de detectar las paredes.
`queue` no se reinicia	Los valores antiguos permanecen; solo `head` y `tail` definen el contenido válido.
`READ_KEY`	Ignora las teclas de función y otras teclas no previstas.
`PRINT_NUM`	Deja tres espacios tras el número impreso.

8.3. Limitaciones de portabilidad y entorno

Cuadro 10: Limitaciones de portabilidad.

Aspecto	Descripción
Plataforma	Diseñado para EMU8086 y DOS; no se garantiza en otros entornos.
Formato COM	Tamaño máximo de 64 KB.
`INT 1Ah`	Su comportamiento puede diferir en hardware real.

8.4. Cosas a tener en cuenta al exponer

Se anticipan las siguientes preguntas durante la exposición:

- **¿Por qué `JAE` y no `JGE`?** Porque `DEC` sobre cero deja `255`, que con signo vale -1 y pasaría la verificación; sin signo, `255` supera el límite.
- **¿Por qué `XOR DX,DX` antes de `DIV`?** Porque la `DIV` de 16 bits divide `DX:AX`; si `DX` conserva basura, el cociente puede desbordar o el resultado ser incorrecto.

- **¿Por qué `CX=2000h` oculta el cursor?** Porque en `INT 10h`, `AH=01h`, el bit 5 de `CH` desactiva el cursor. (Ubicación exacta en el código por confirmar.)
- **¿Por qué `PUSH AX` dos veces en `DRAW_CELL`?** Respuesta por confirmar contra el listado: debe explicar qué valor protege cada `PUSH` frente al `DIV` de 8 bits y a `INT 10h`.
- **¿Por qué `SPAWN_FOOD` no se cuelga con el tablero lleno?** Porque el bucle con `LOOP` tiene un máximo de 1000 intentos y luego retorna.
- **¿Por qué la cola es circular y no se desplazan datos?** Para evitar el `memmove`: solo cambian los índices `head` y `tail`.
- **¿Qué bandera decide `JE/JNE` y cuál `JB/JAE`?** `ZF` decide `JE/JNE`; `CF` decide `JB/JAE`.

8.5. Posibles mejoras futuras

Entre las mejoras posibles, que *no* forman parte del programa actual, se cuentan: velocidad progresiva, reinicio con la tecla `R`, récord en disco, cabeza diferenciada del cuerpo, sonido con `INT 10h AH=0Eh`, pausa con la tecla `P` y tablero configurable mediante `EQU`.

9. Pruebas y validación

9.1. Lista de verificación

[ ] La comida aparece en posiciones distintas entre partidas.

[ ] Las flechas y las teclas WASD (mayúsculas y minúsculas) cambian la dirección.

[ ] El giro de 180° es rechazado.

[ ] El choque con la pared funciona en los cuatro lados.

[ ] El choque con el propio cuerpo termina la partida.

[ ] Al comer, la serpiente crece un segmento.

[ ] El puntaje se muestra correctamente con 0, 1, 2 y 3 cifras.

[ ] `ESC` sale del programa y restaura el cursor.

[ ] El borde del tablero se mantiene intacto durante el juego.

[ ] Se muestra “GAME OVER!” con el puntaje final.

[ ] No hay parpadeo de pantalla completa.

9.2. Procedimiento de prueba recomendado

1. Abrir `snake.asm` en EMU8086 y ensamblarlo sin errores.
2. Ejecutar el programa en el emulador y comprobar que se dibuja el borde, la serpiente inicial y la comida.
3. Jugar durante unos 30 segundos probando flechas y WASD.
4. Verificar el rechazo del giro de 180°.
5. Provocar choques con cada una de las cuatro paredes y con el cuerpo.
6. Comer varias veces y observar el crecimiento y el puntaje con una, dos y tres cifras.
7. Verificar `GAME_OVER` y la espera de tecla.
8. Repetir y salir con `ESC`; confirmar que el cursor y el modo de video se restauran.
9. Marcar cada ítem de la lista de verificación.

10. Anexos

10.1. Código fuente completo (snake.asm)

El archivo `snake.asm` no está disponible en esta carpeta; por ello, no se puede incluir el código fuente en este anexo.

10.2. Tabla de variables y estructuras

La Tabla **??** repite la Tabla **??** en formato de varias páginas.

Cuadro 11: Constantes y variables (Anexo B).

Nombre	Tipo	Tamaño	Uso
`BOARD_W`	`EQU`	—	Ancho: 40.
`BOARD_H`	`EQU`	—	Alto: 10.
`BOARD_SIZE`	`EQU`	—	Celdas: 400.
`board`	`DB`	400 bytes	Estado de celdas (0 libre, 1 serpiente, 2 comida).
`queue`	`DW`	400 palabras	Cola circular de índices.
`head`	`DW`	1 palabra	Índice de escritura.
`tail`	`DW`	1 palabra	Índice de lectura.
`dir`	`DB`	1 byte	Dirección (0 a 3).
`headRow`	`DB`	1 byte	Fila de la cabeza.
`headCol`	`DB`	1 byte	Columna de la cabeza.
`score`	`DW`	1 palabra	Puntaje.
`seed`	`DW`	1 palabra	Semilla del generador.
`delayTicks`	`DB`	1 byte	Ticks de espera (2).
`msgScore`	`DB`	cadena	“Puntaje: ”.
`msgGameOver`	`DB`	cadena	“GAME OVER!”.

10.3. Tabla de servicios BIOS/DOS usados

La Tabla **??** lista los servicios del sistema. Las funciones marcadas con asterisco están pendientes de confirmar contra el código fuente.

Cuadro 12: Servicios BIOS y DOS empleados.

INT	AH	Función	Dónde se usa
`10h`	`01h`	Fijar forma del cursor (`CX=0607h` restaura; `CX=2000h` oculta).	Salida del programa; inicio (por confirmar).
`10h`	`00h`*	Fijar modo de video.	Inicio y salida.
`10h`	`02h`*	Posicionar el cursor.	`DRAW_CELL`, `DRAW_BORDER`.
`10h`	`09h`*	Escribir carácter con atributo.	`DRAW_CELL`.
`16h`	`01h`	Consultar teclado sin bloquear.	`READ_KEY`.
`16h`	`00h`	Leer y consumir tecla.	`READ_KEY`, `GAME_OVER`.
`1Ah`	`00h`	Leer contador de ticks.	`DELAY`.
`21h`	*	Salida de caracteres o cadenas.	`PRINT_NUM`, mensajes.

10.4. Referencias

Referencias

[1] EMU8086 Microprocessor Emulator, *Manual de usuario*. (Edición y versión por confirmar.)

[2] Brown, R. *Ralf Brown's Interrupt List* (INT 10h, INT 16h, INT 1Ah, INT 21h).

[3] Universidad Nacional de San Agustín de Arequipa, Escuela Profesional de Ingeniería de Sistemas. *Guía de prácticas de Arquitectura de Computadoras*. (Edición por confirmar.)

Notas para el autor

Los siguientes puntos requieren confirmación humana antes de la entrega:

1. El archivo `snake.asm` no estaba disponible al redactar este documento. El Anexo A usa `\lstinputlisting{snake.asm}`: coloque el archivo en la misma carpeta o pegue su contenido literal en un `lstlisting`.
2. La versión exacta de EMU8086 no se indicó en la fuente (Tabla **??**).
3. La lista exacta de registros modificados y preservados con `PUSH/POP` de cada procedimiento no estaba definida (Tabla **??**).
4. Los números de función `AH` marcados con asterisco en la Tabla **??**, la forma exacta de terminar el programa y las funciones de `INT 21h` deben verificarse en el código.
5. El lugar del código donde se aplica `CX=2000h` y la razón del doble `PUSH AX` en `DRAW_CELL` deben confirmarse antes de la exposición.
6. Las marcas de tiempo de la micro-traza son ilustrativas; los valores de las variables sí provienen de la especificación.
7. Debe confirmarse cómo se inicializa `seed` (afecta la limitación "semilla repetible").
8. Las referencias bibliográficas requieren edición, año y versión definitivos; las prácticas del curso se nombraron según el enunciado proporcionado.